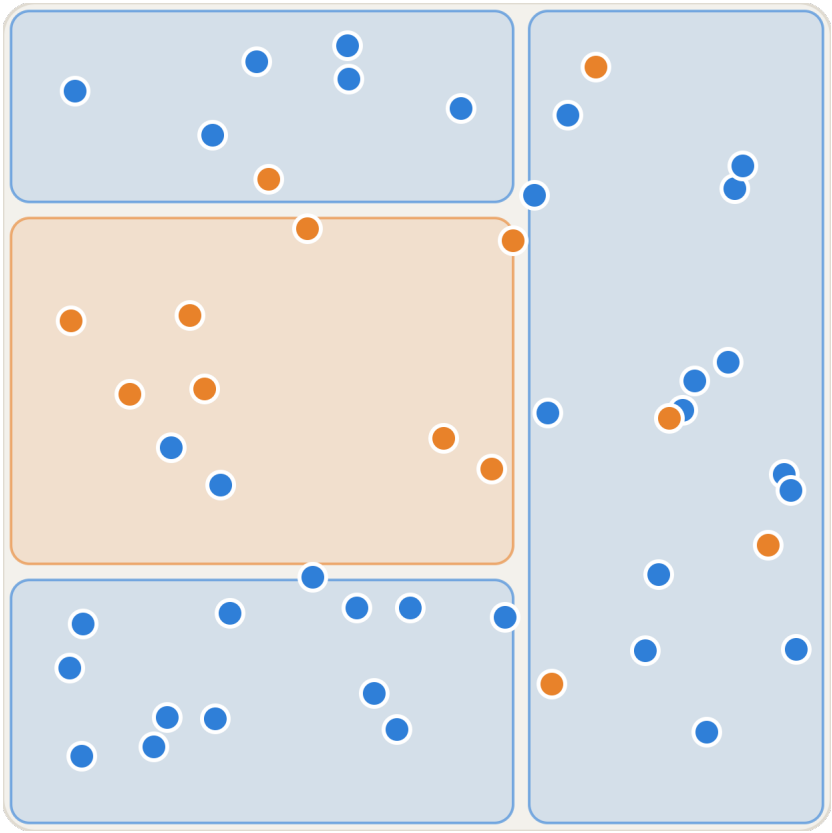
Lecture 1 of 5 · From decision trees to LLMs

What does it mean for a machine to learn?

Decision trees, random forests, gradient boosting and XGBoost, explained from scratch

Paras Koundal



Note

All the graphics presented here are animations.

If a graphic shows up as empty here, check the mov folder on github.com/ParasKoundal/ML-Slides to see the corresponding video. The original recording of the presentation is only available on request.

Five lectures, one question: how does a model learn?

1

Trees & ensembles

Learning as search · random forests · boosting · XGBoost

2

Gradients & neural networks

Loss functions · gradient descent · backpropagation

3

Training deep networks

Optimisers · normalisation · CNNs · embeddings

4

Attention & Transformers

Self-attention · next-token prediction

5

From Transformer to LLM

Scale · sampling · fine-tuning · RLHF

Key idea. Today the learning happens by **searching** and **averaging**, which you can watch step by step. At the very end of today, boosting introduces **gradients**, the engine of every later lecture.

Traditional programming vs machine learning

Traditional programming

A person writes the rules.

rules + data → answers

Example: an engineer writes “if pH < 6.8, raise an alarm.” The rule is only as good as the engineer’s knowledge.

Machine learning

The computer finds the rules from examples.

data + answers → rules

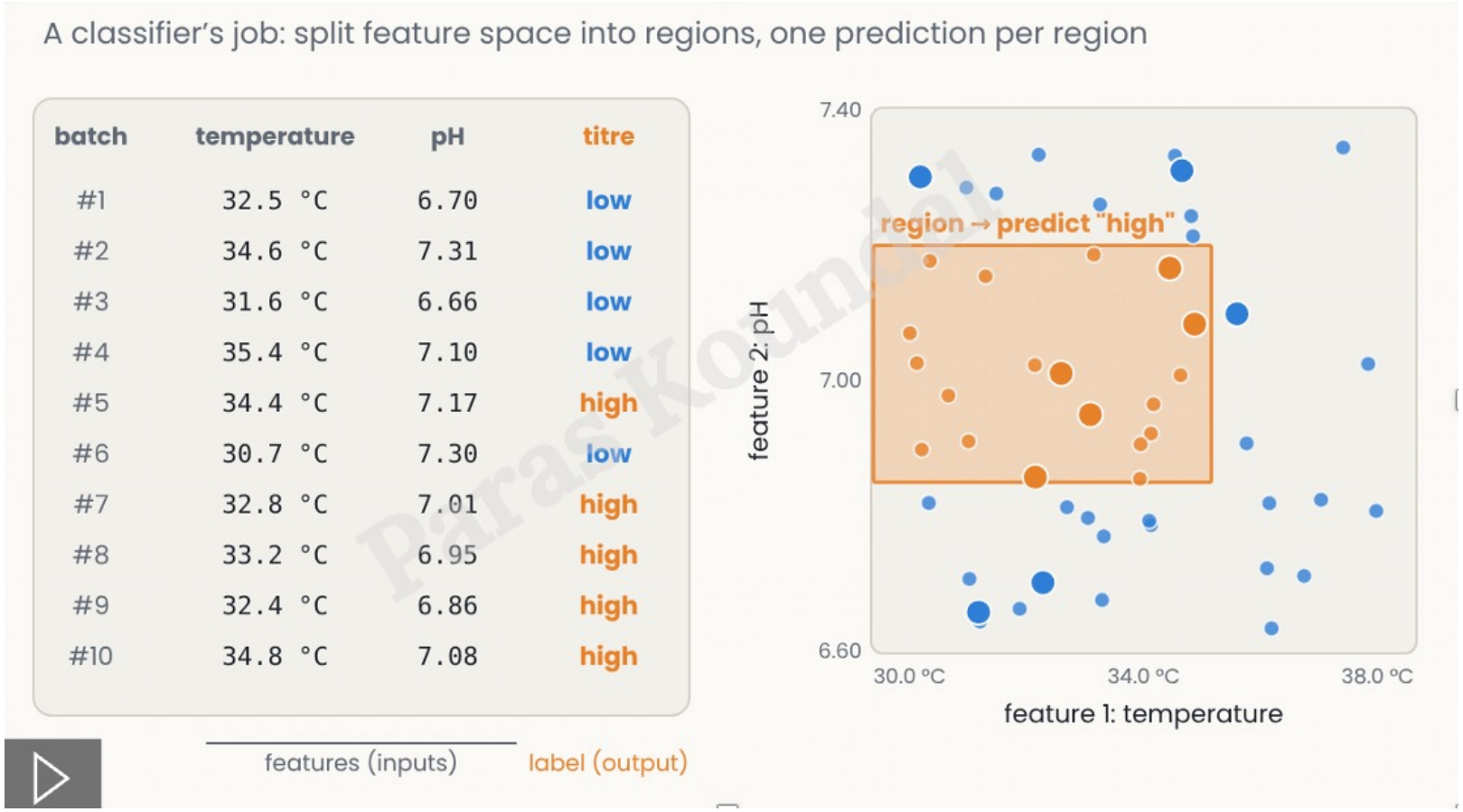
Example: give it 500 past batches with their measured conditions and outcomes; it works out which conditions led to good outcomes.

Key idea. A machine-learning model is a set of rules (a function) whose details are **not written by hand** but **adjusted automatically** until it agrees with known examples.

Vocabulary we will use all lecture

term	meaning (with a bioprocess flavour)
Example / sample	one row of the data: here, one batch
Feature (input, x)	a measured quantity used to predict: temperature, pH, glucose feed
Label (target, y)	the answer we want to predict: high or low titre, or a number
Model, f(x)	the function that turns features into a prediction
Parameters	the parts of the model that training adjusts (for a tree: its questions and thresholds)
Hyper-parameters	settings chosen by us before training, e.g. the maximum tree depth

A data table becomes points in feature space



Key idea. From now on, "a data set" means a cloud of coloured points, and "a model" means a way of colouring the whole space.

Every method answers the same three questions

1

Model

Which family of functions are we allowed to choose from?

Today: trees, which cut space into boxes

2

Loss

How do we measure how wrong one particular choice is?

Today: impurity, squared error, log-loss

3

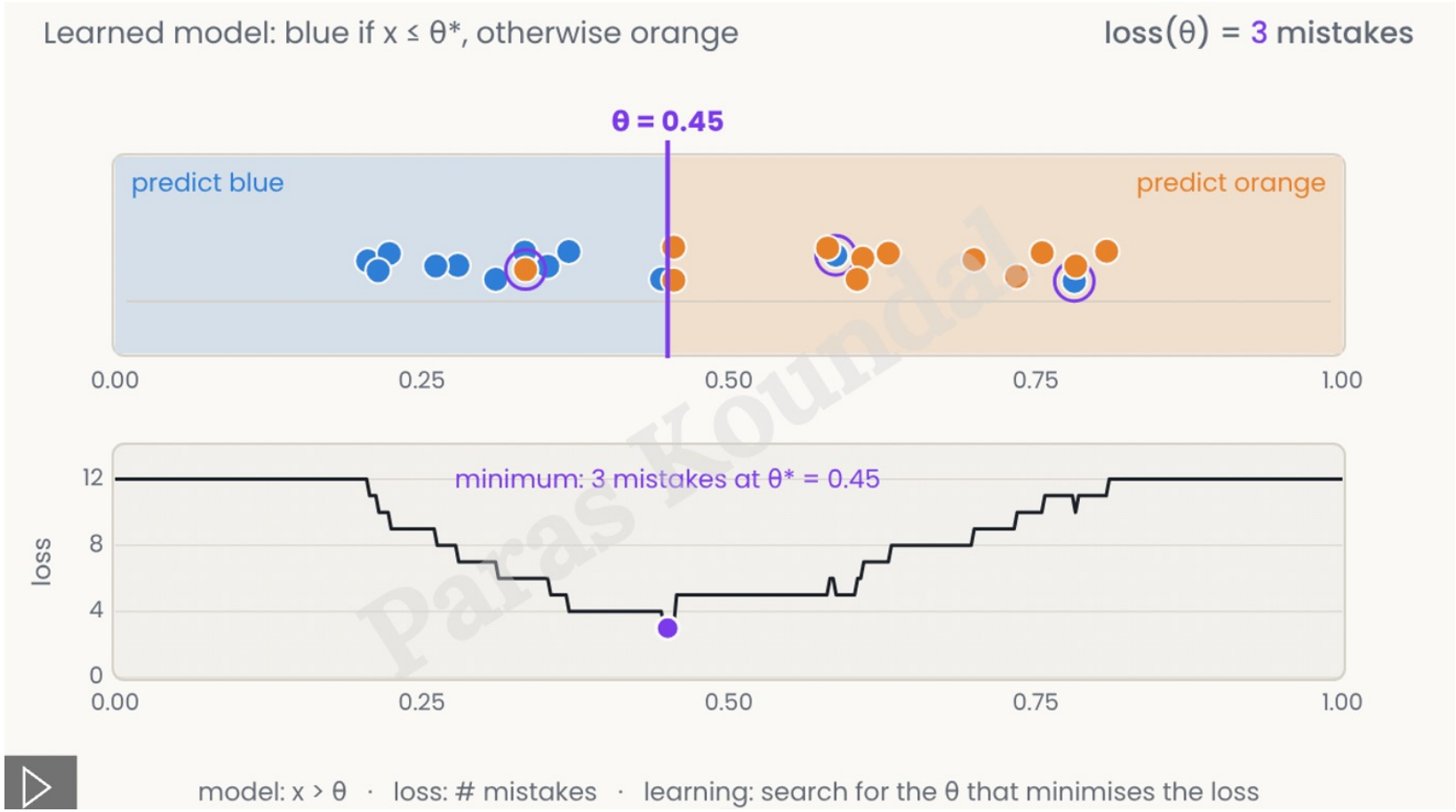
Learning

How do we move to a better choice?

Today: greedy search, averaging, sequential correction

Key idea. **Training** = using the learning procedure (3) to find the model (1) with the smallest loss (2). This sentence is true from decision trees all the way to ChatGPT.

The simplest possible learning: one threshold



Paras Koundal

Key idea. Learning = searching for the parameter value with the lowest loss. Trees do this search exhaustively; neural networks (Lecture 2) do it with gradients.

The real goal: good predictions on new data



Training set · ~60–70%

Used to fit the parameters: which questions the tree asks.

Validation set · ~10–20%

Used to choose hyper-parameters, e.g. how deep the tree may grow.

Test set · ~20%

Locked away; used once at the end for an honest score.

Key idea. A model that memorises the training set can score 100% there and still fail on the next batch. This is called **overfitting**. We only trust scores measured on data the model never saw.

Analogy: a student who memorises last year's exam answers gets full marks on that paper, but has learned nothing for a new paper.

Check your understanding

Q1

In a data set of past batches, what is a feature and what is the label?

Q2

What is the difference between a parameter and a hyper-parameter?

Q3

Why can’t we judge a model by its accuracy on the training data?

Q4

In the threshold animation, what exactly was being searched, and what was being minimised?

01

Decision trees

Learning by asking yes/no questions



Cutest cat?

What's a cat?





























Can Fly

No

Yes

Taller than me
(baby)

Yes

No

Says meow

No

Yes

A decision tree is a flowchart of yes/no questions

Is pH ≤ 7.24?

no → **predict low**

yes → Is pH ≤ 6.78?

yes → **predict low**

no → Is T ≤ 35.1 °C?

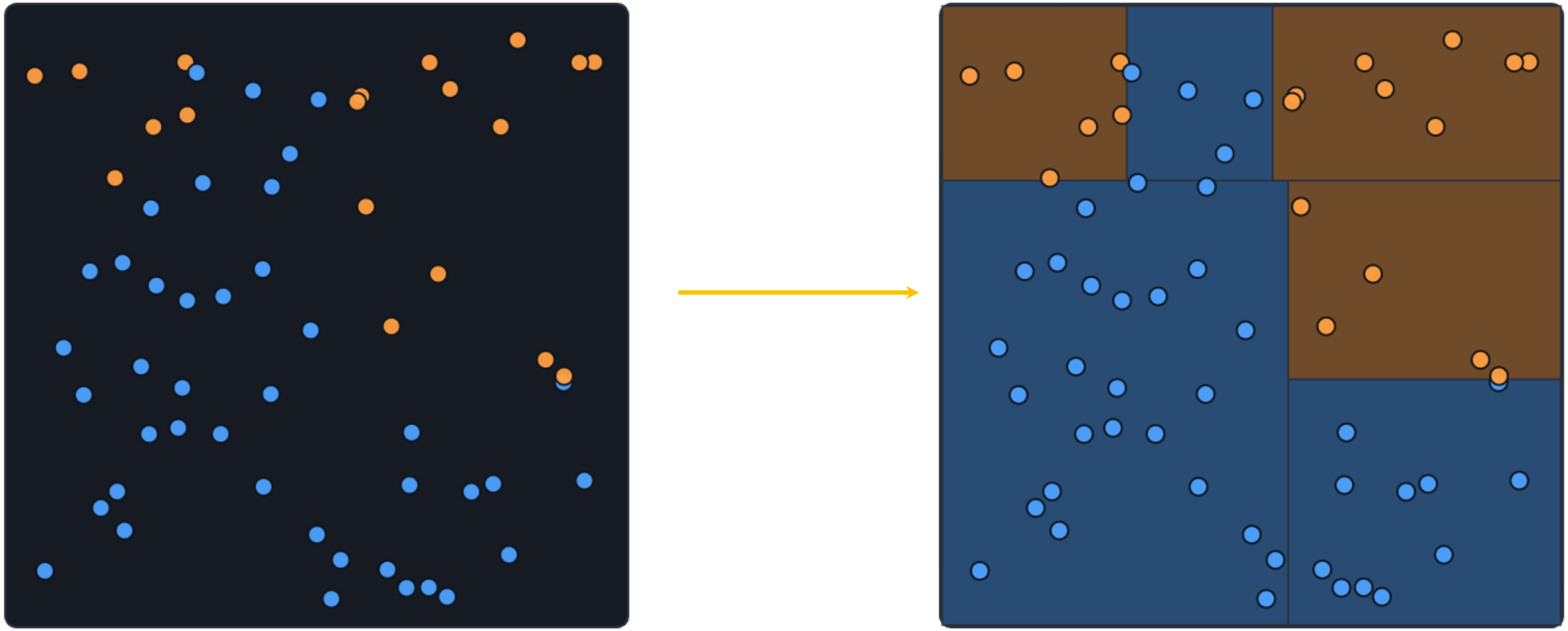
yes → **predict high**

no → **predict low**

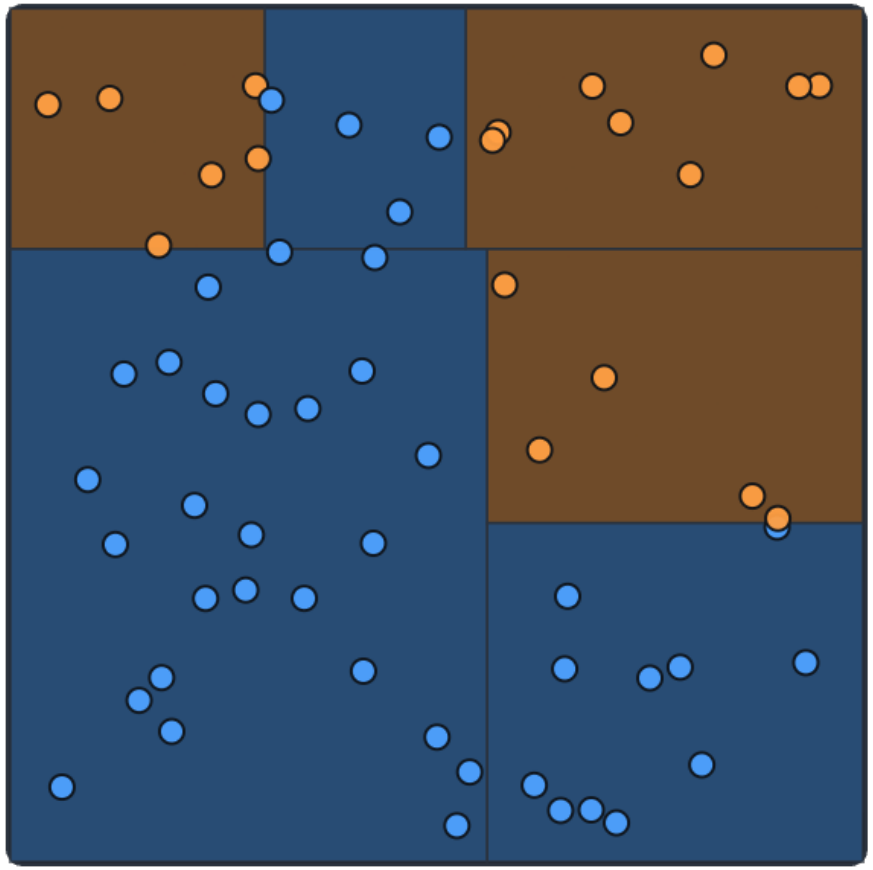
part	meaning
Root	the first question, asked of every sample
Internal node	a question of the form “is feature ≤ threshold?”
Branch	the yes or no answer, leading to the next node
Leaf	the end of a path: stores a prediction
Depth	the number of questions on the longest path

Key idea. Each question looks at **one feature** and compares it with **one threshold**. That is why a tree’s regions are boxes with edges parallel to the axes.

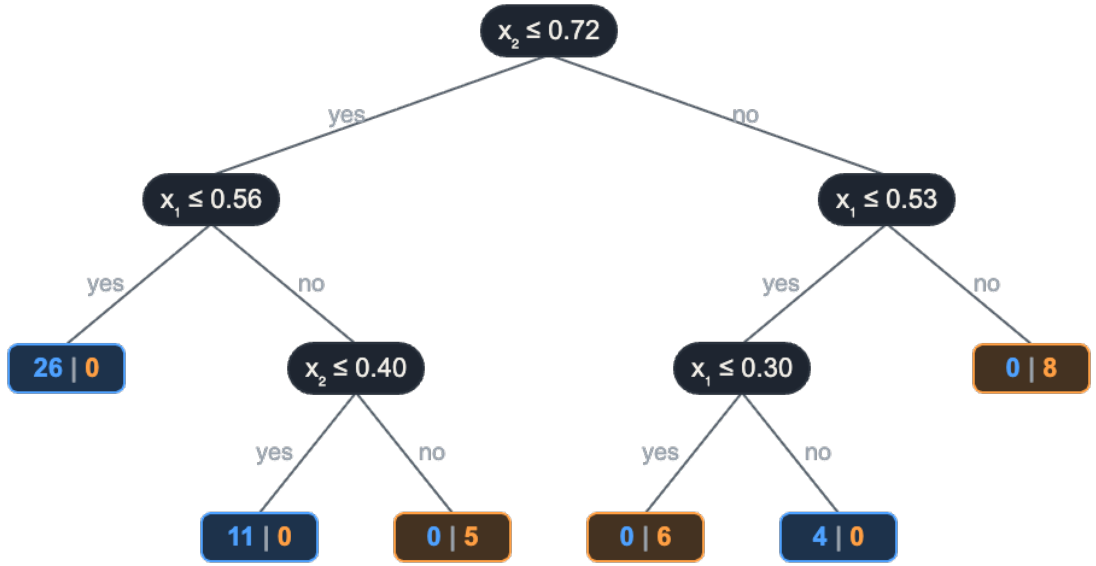
A decision tree is a sequence of yes/no questions



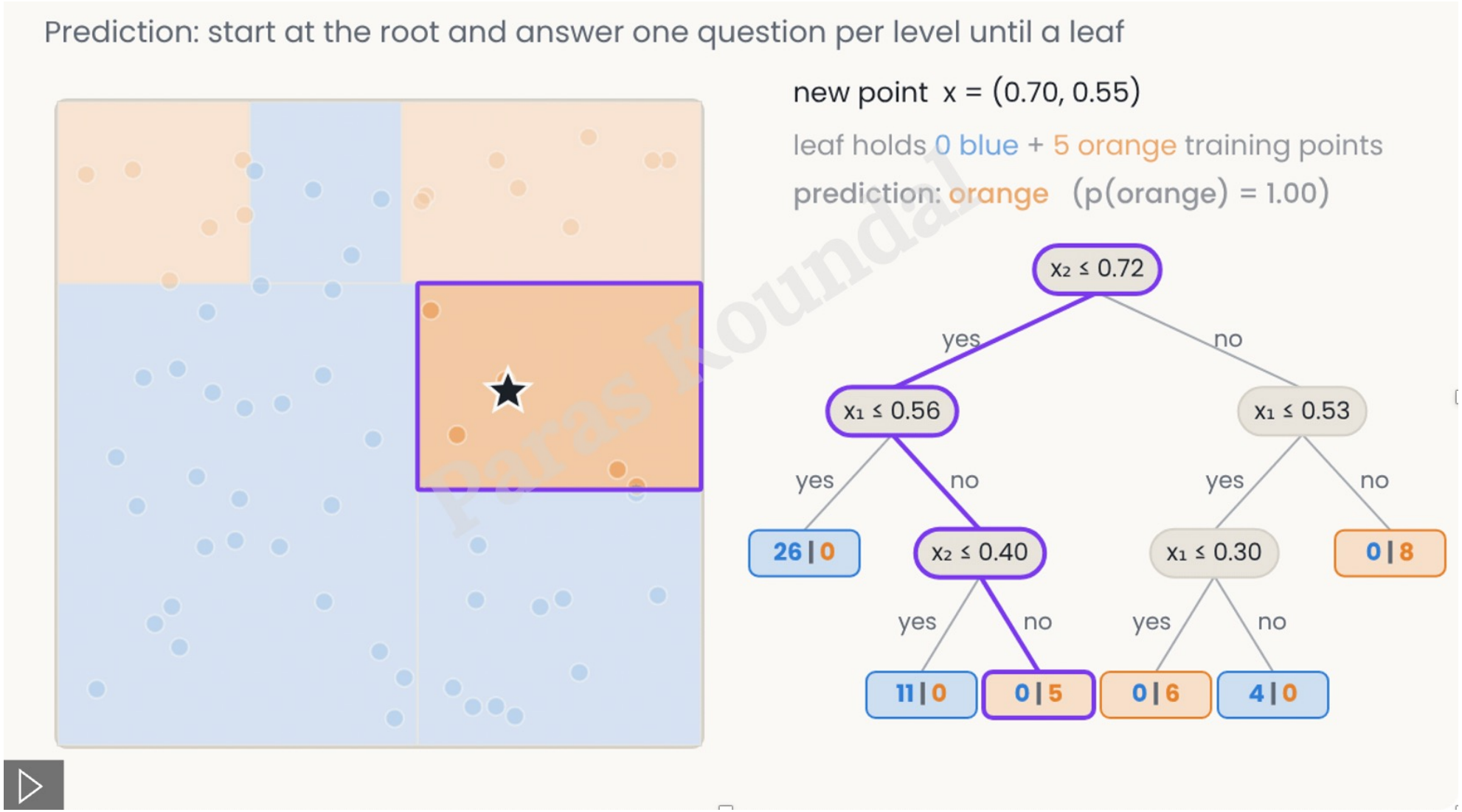
A decision tree is a sequence of yes/no questions



=



Making a prediction with a trained tree



Paras Koundal

Building a tree by hand: 10 batches

batch	T (°C)	pH	titre
#1	32.5	6.70	low
#2	34.6	7.31	low
#3	31.6	6.66	low
#4	35.4	7.10	low
#5	34.4	7.17	high
#6	30.7	7.30	low
#7	32.8	7.01	high
#8	33.2	6.95	high
#9	32.4	6.86	high
#10	34.8	7.08	high

The question we must answer

Which **first** question best separates the high-titre batches from the low ones?

Candidate questions

Any feature, any threshold. Only thresholds **between** two neighbouring values matter, e.g. pH ≤ 6.78 lies between 6.70 and 6.86.

Key idea. Before we can pick the best question, we need a number that says how **mixed** a group of batches is. That number is called **impurity**.

Impurity: a number for “how mixed is this group?”

Gini impurity (two classes)

$$\text{Gini} = 2 \cdot p \cdot (1 - p)$$

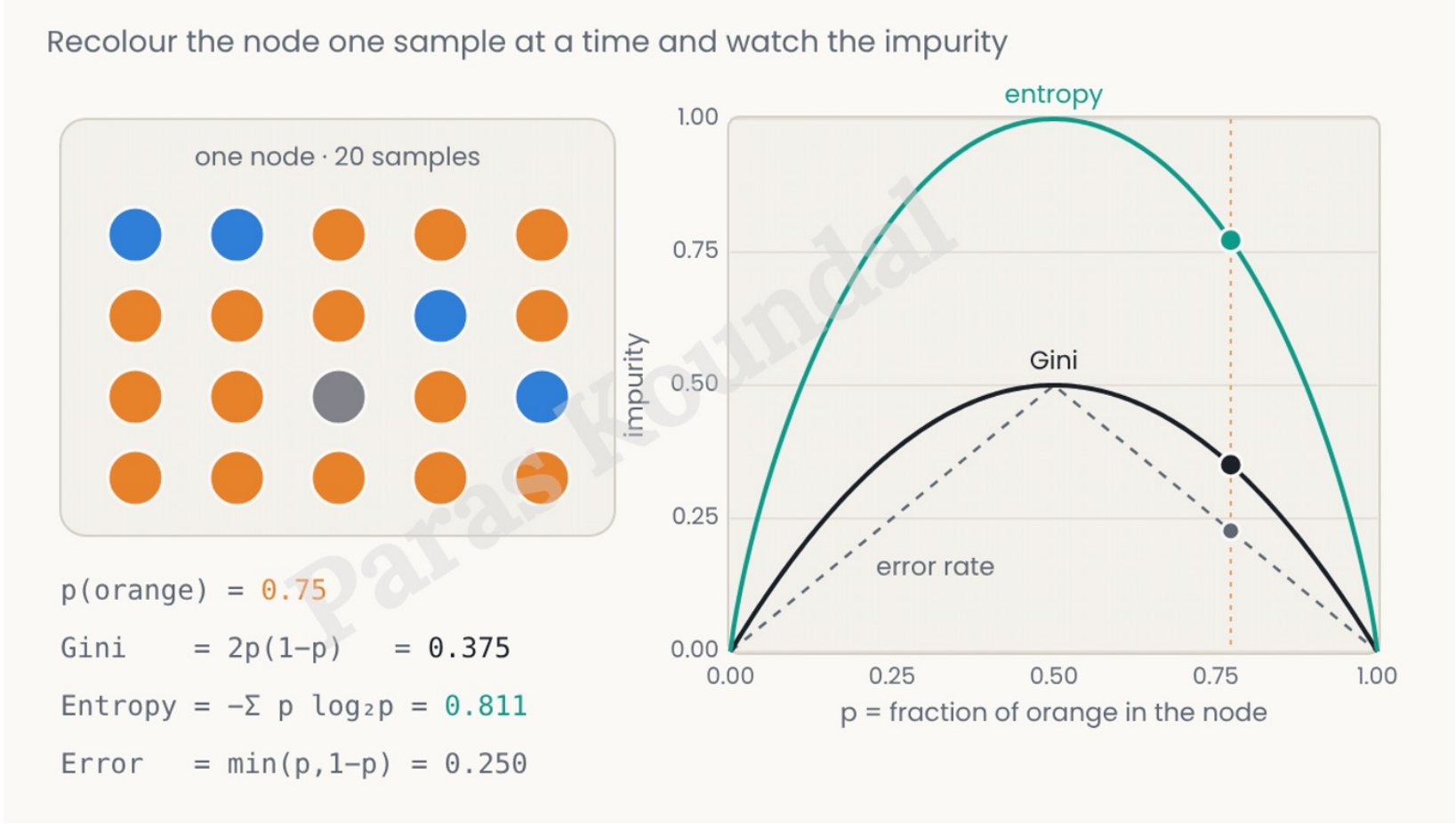
p = fraction of the group that is “high”.

Meaning: pick two batches at random from the group; Gini is the chance they have **different** labels.

group	p	Gini
5 high, 5 low	0.50	2·0.5·0.5 = 0.500
5 high, 3 low	0.625	2·0.625·0.375 = 0.469
5 high, 1 low	0.83	2·0.83·0.17 = 0.278
2 low, 0 high	0	0 (pure)

Key idea. Gini is **0** for a pure group (all one label) and largest, **0.5**, for a 50/50 mix. A good question creates purer groups.

Gini and entropy as a group becomes more mixed



Key idea. Gini and entropy are preferred over plain error rate because their curvature rewards **any** improvement in purity, even when the majority class does not change.

Scoring one question: the gain

parent: all 10

5 low · 5 high

Gini 0.500

→

pH ≤ 6.78 → yes (n = 2)

2 low · 0 high

Gini 0.000

pH ≤ 6.78 → no (n = 8)

3 low · 5 high

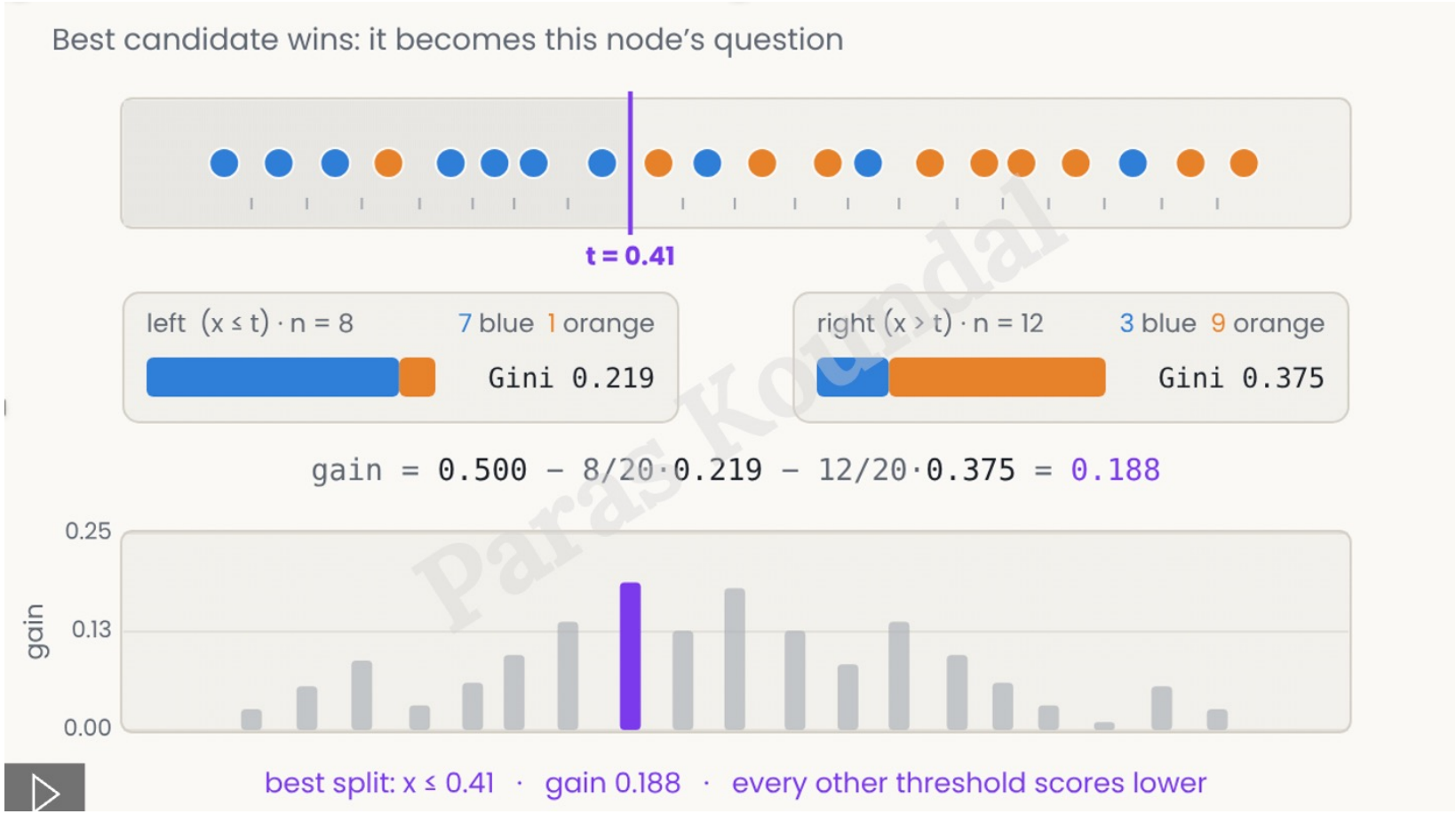
Gini 0.469

gain = Gini(parent) − (2/10)·Gini(yes) − (8/10)·Gini(no)

= 0.500 − 0.2·0.000 − 0.8·0.469 = 0.125

Key idea. Gain = how much impurity the question removes. Each child is weighted by its size, so peeling off one lucky point cannot win on its own.

How the computer checks every threshold



Key idea. Sorting first makes the scan cheap: moving the threshold one step moves exactly one sample from right to left, so counts update instantly.

From a random first question to a learned tree

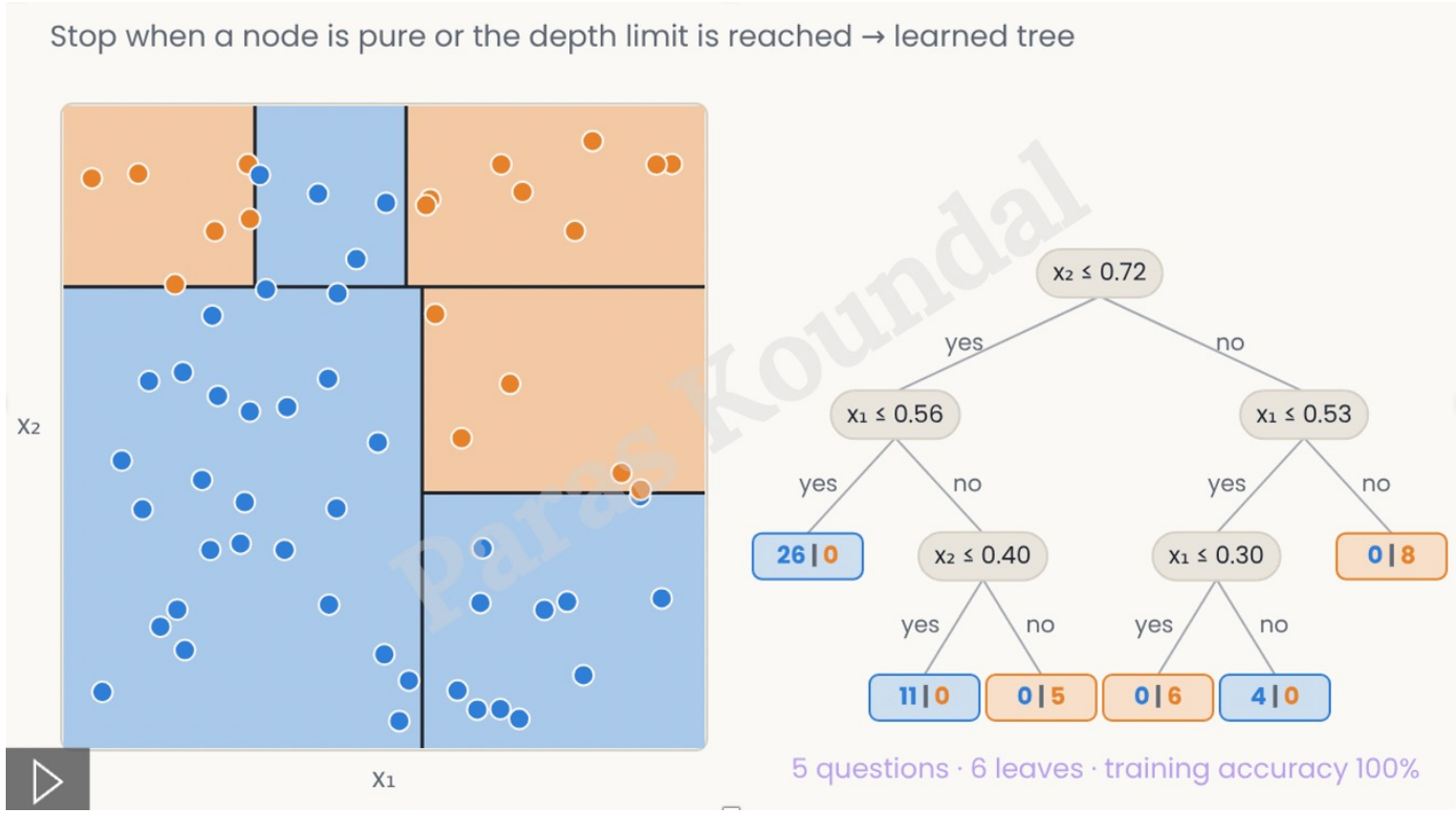
(1) A random first question ($x_1 \le 0.80$) splits the data but barely separates the colours: gain ≈ 0.02.

(2) The algorithm instead tries every threshold on x_1, then on x_2. Top right: the candidate being tried and the best so far.

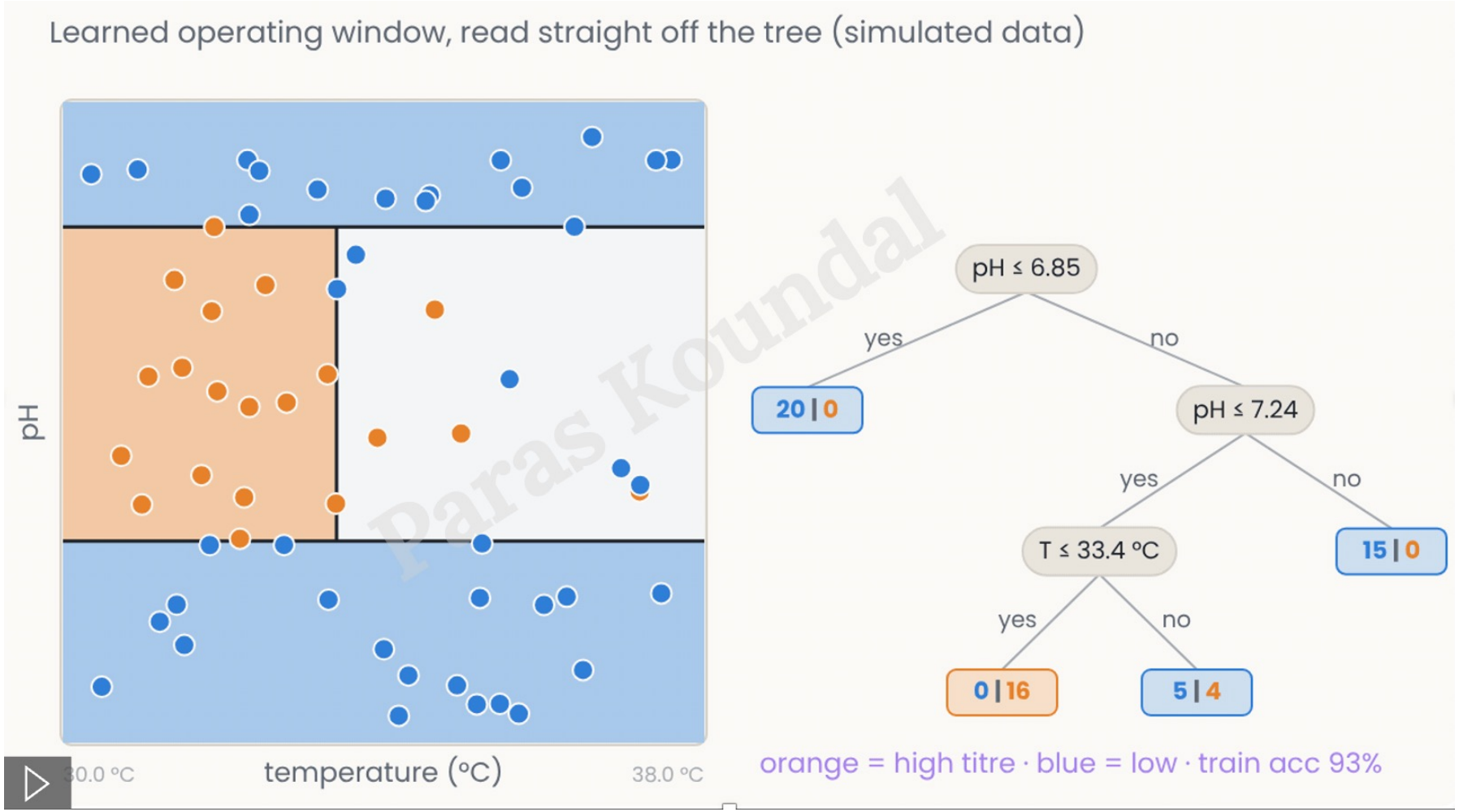
(3) The winner ($x_2 \le 0.72$) is locked in and appears in the tree diagram.

(4) The same search repeats inside each child region (red box) until groups are pure or depth = 3.

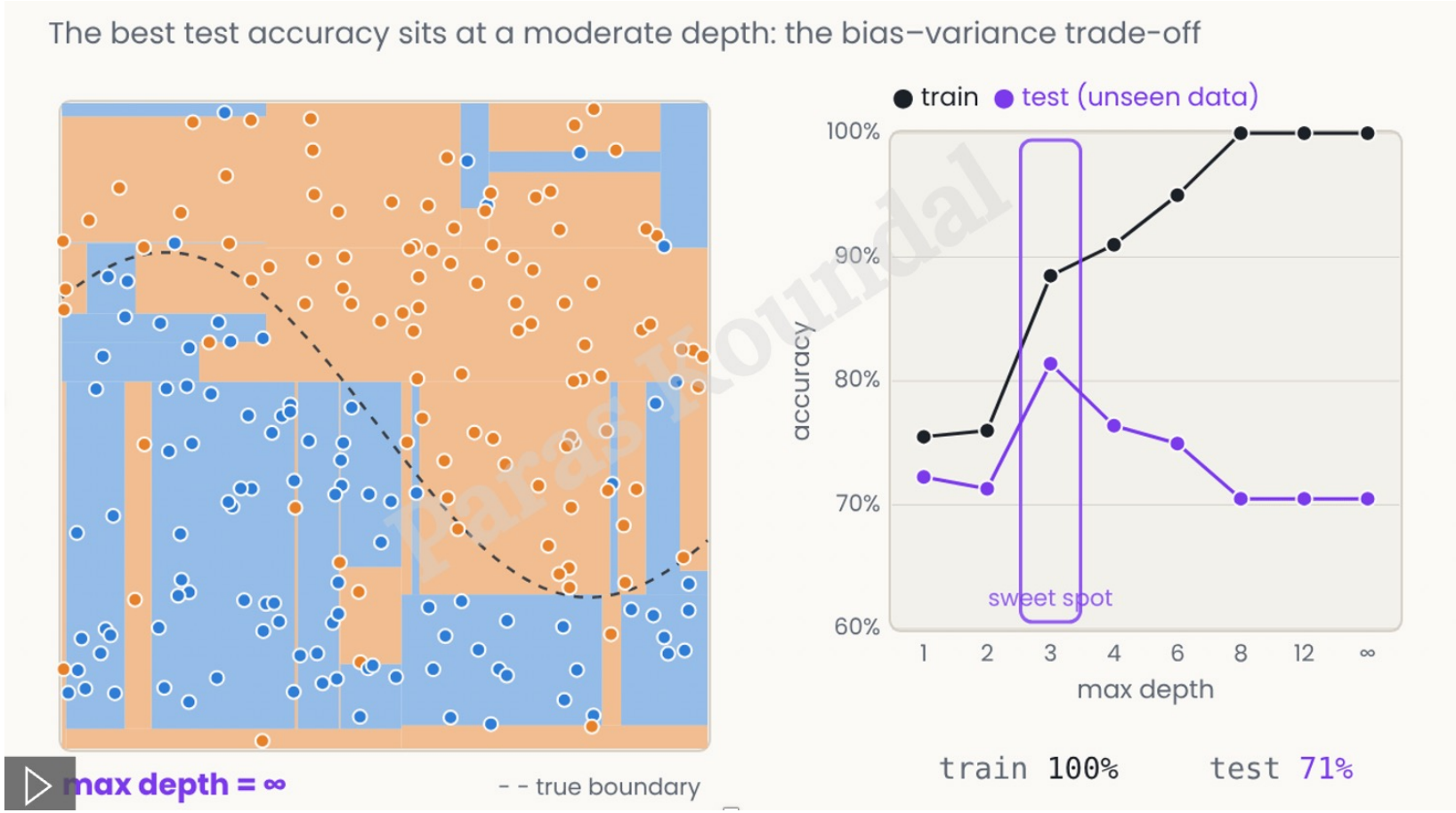
Key idea. A tree is learned by **repeating one simple step**: in the current group, find the single question with the largest gain.



Bioprocess example: an operating window from past batches



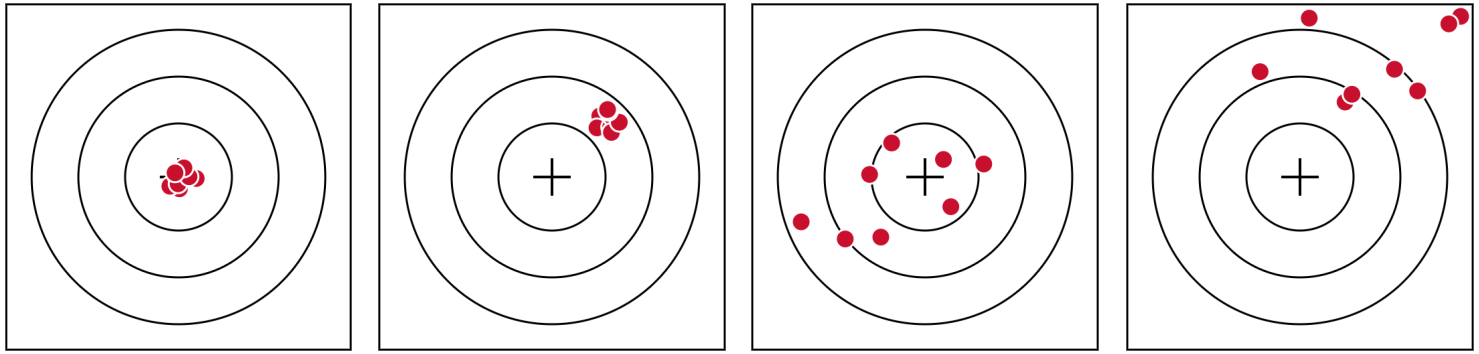
Deeper is not better: overfitting



Parag Kourdal

Key idea. Too simple = **high bias** (underfit). Too complex = **high variance** (overfit). Choose depth using the **validation** set.

Bias and variance, in pictures



low bias, low variance | high bias, low variance | low bias, high variance | high bias, high variance

Bias: the model is systematically off, because it is too simple to represent the truth (a shallow tree).

Variance: the model changes a lot when trained on a slightly different data set (a deep tree).

Trees can also predict numbers (regression trees)

Classification tree

- label is a category (high / low)
- impurity = Gini or entropy
- leaf predicts the majority class

Regression tree

- label is a number (titre in g/L)
- impurity = squared error around the group mean
- leaf predicts the **mean** of its samples

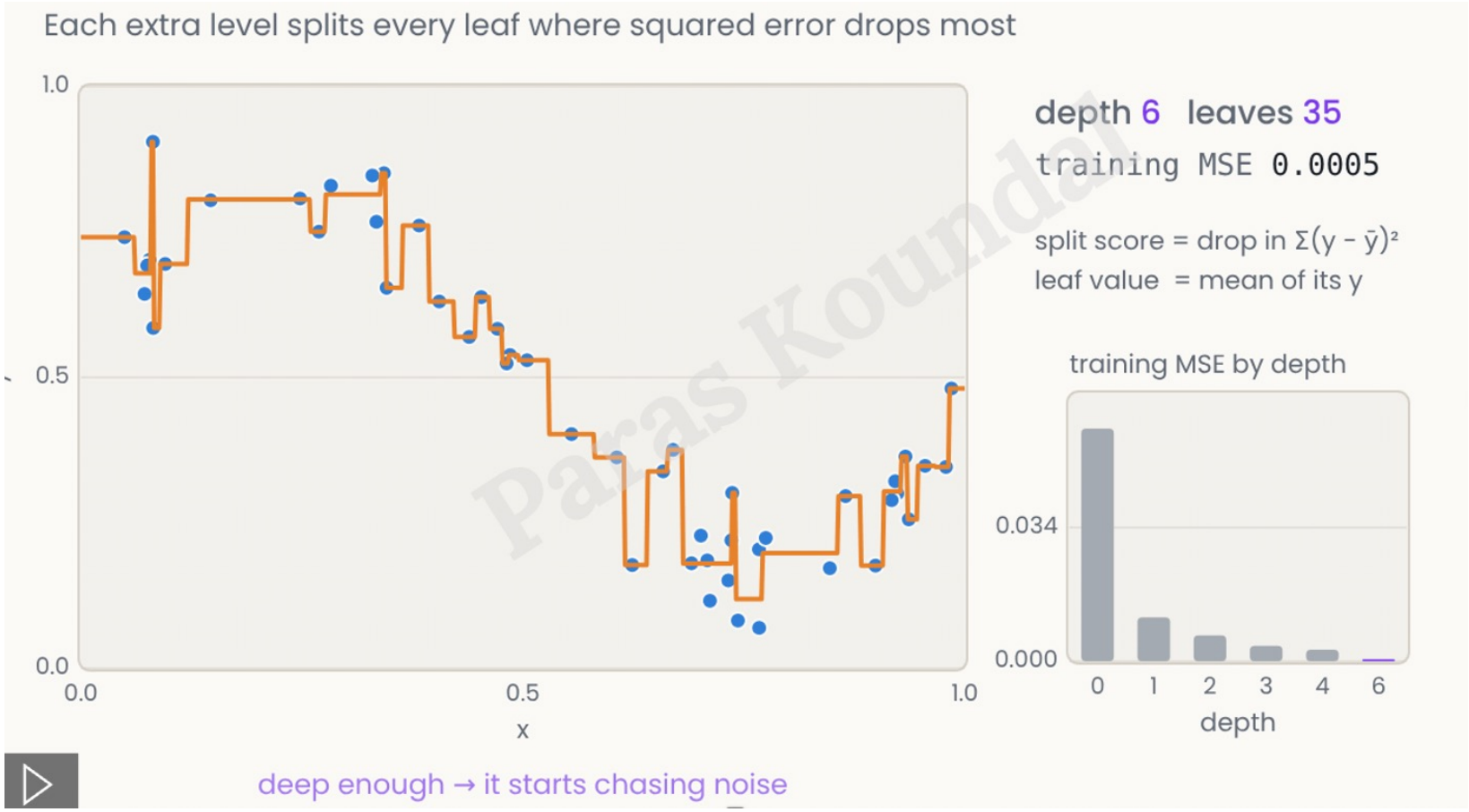
Squared error by hand

A leaf holds titres 2, 4 and 6 g/L. Mean = 4, so the leaf predicts 4.

squared error = $(2-4)^2 + (4-4)^2 + (6-4)^2 = 4 + 0 + 4 = 8$

A split is good if the two children's squared errors add up to much less than the parent's.

A regression tree fitting a curve, depth by depth



Key idea. A regression tree is a **step function**. More depth = more steps = closer fit to the training data, including its noise.

Hyper-parameters that keep a tree in check

setting (scikit-learn name)	what it does	direction
max_depth	maximum number of questions on any path	smaller → simpler
min_samples_leaf	every leaf must contain at least k samples	larger → simpler
min_impurity_decrease	only split if the gain is at least this big	larger → simpler
max_leaf_nodes	cap the total number of leaves	smaller → simpler
ccp_alpha	grow fully, then prune back branches that are not worth their extra leaves	larger → simpler

Key idea. All of these trade a little training accuracy for better accuracy on new data. Choose their values by comparing scores on the **validation** set.

02

Random forests

Many noisy trees, one stable answer



Intuition: the wisdom of crowds

Guess the number of beans in a jar

One person's guess can be far off. The **average** of 100 independent guesses is usually much closer to the truth.

Why? Individual errors point in different directions and cancel when averaged.

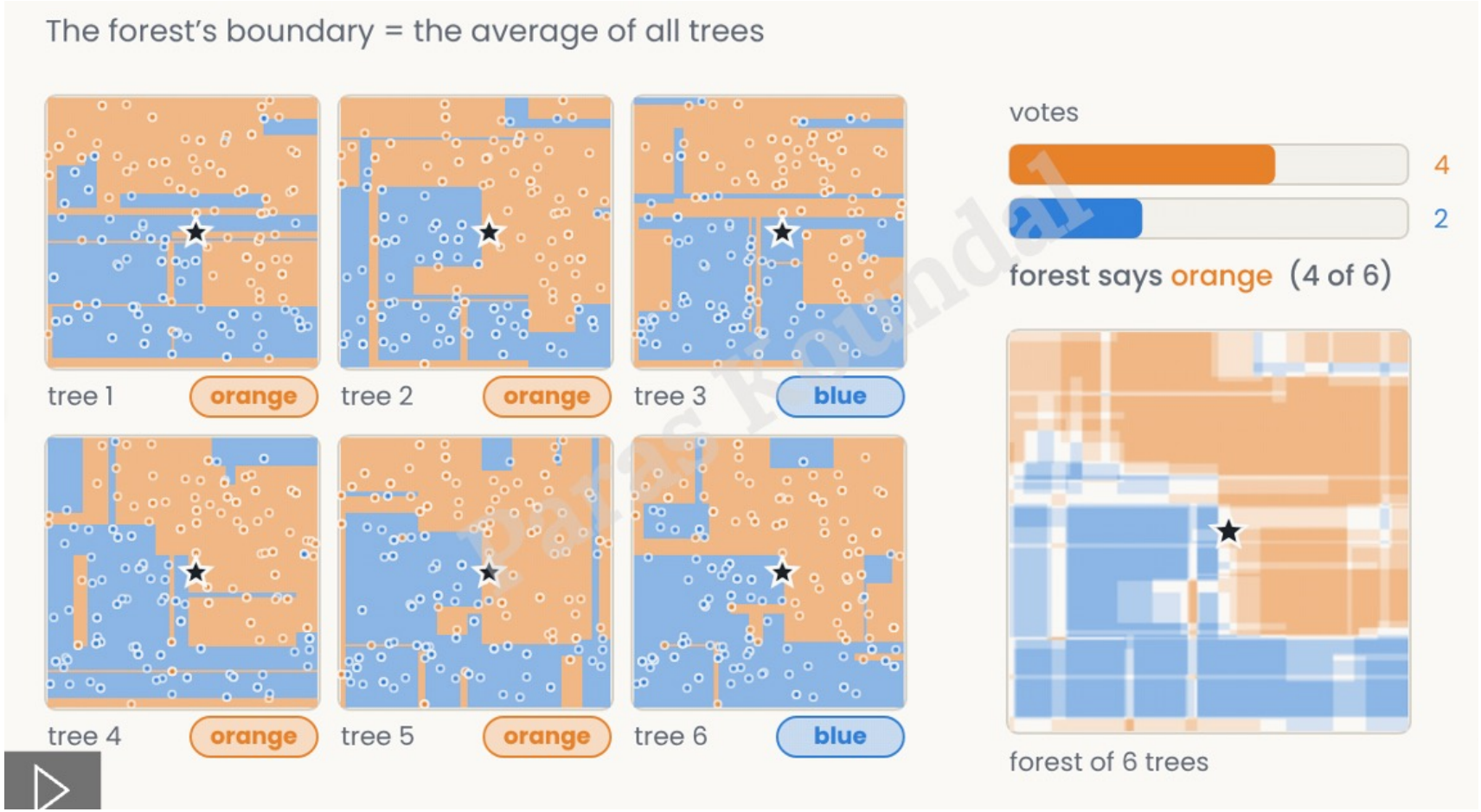
The catch

If everyone copies the same loud person, their errors are **the same** and nothing cancels.

Averaging helps only when the errors are **independent** (different from each other).

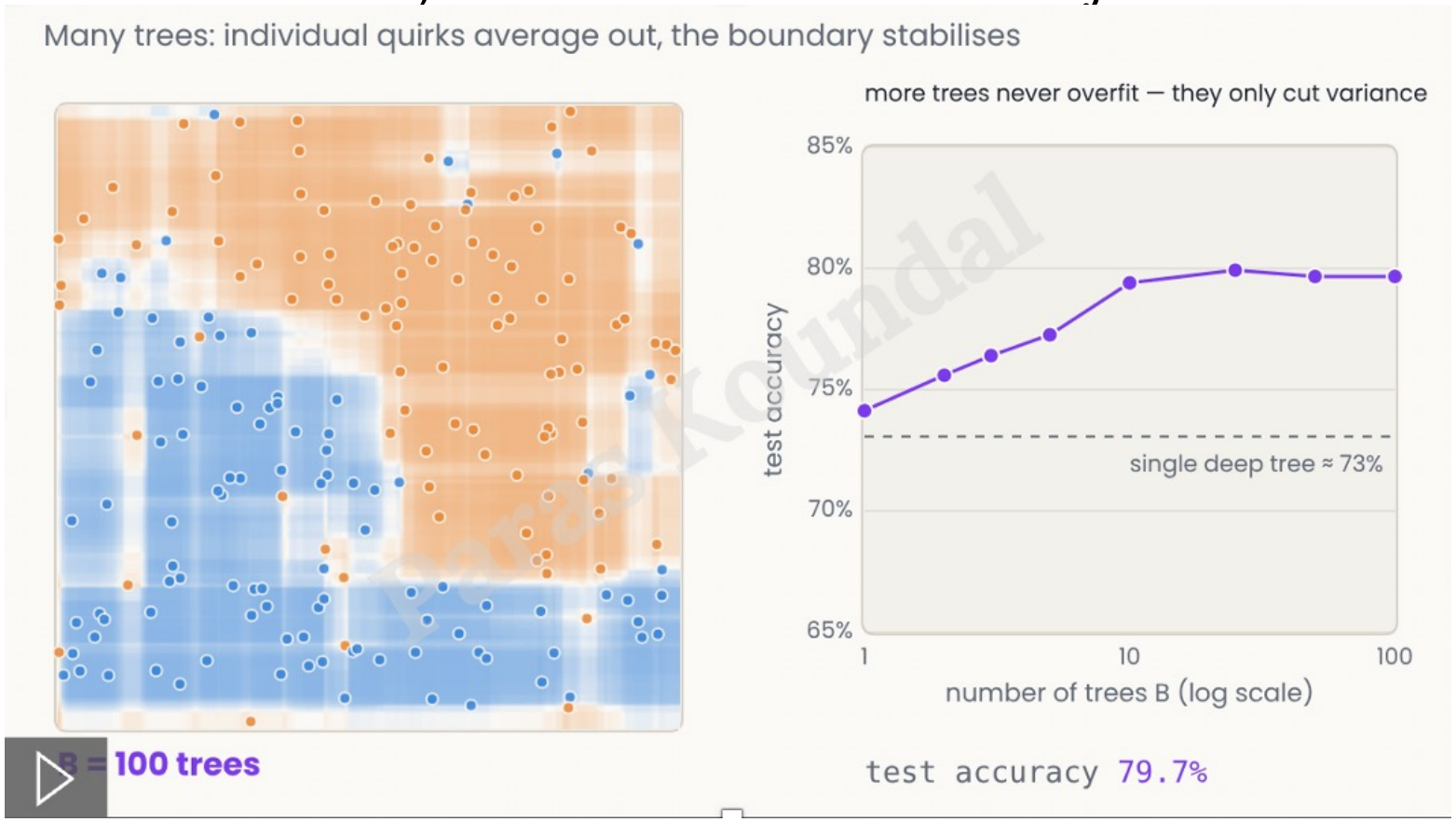
Key idea. A random forest = a crowd of deep trees, deliberately made different from each other, whose predictions are averaged.

Training and using a random forest



Key idea. Classification: **majority vote** (or average probability). Regression: **average** of the trees' numbers. Trees are independent, so they train in parallel.

More trees: a smoother, more reliable boundary



Key idea. Adding trees does **not** cause overfitting in a random forest. Use enough trees that the OOB or validation score has flattened (often a few hundred).

Bioprocess example: which process variables matter?

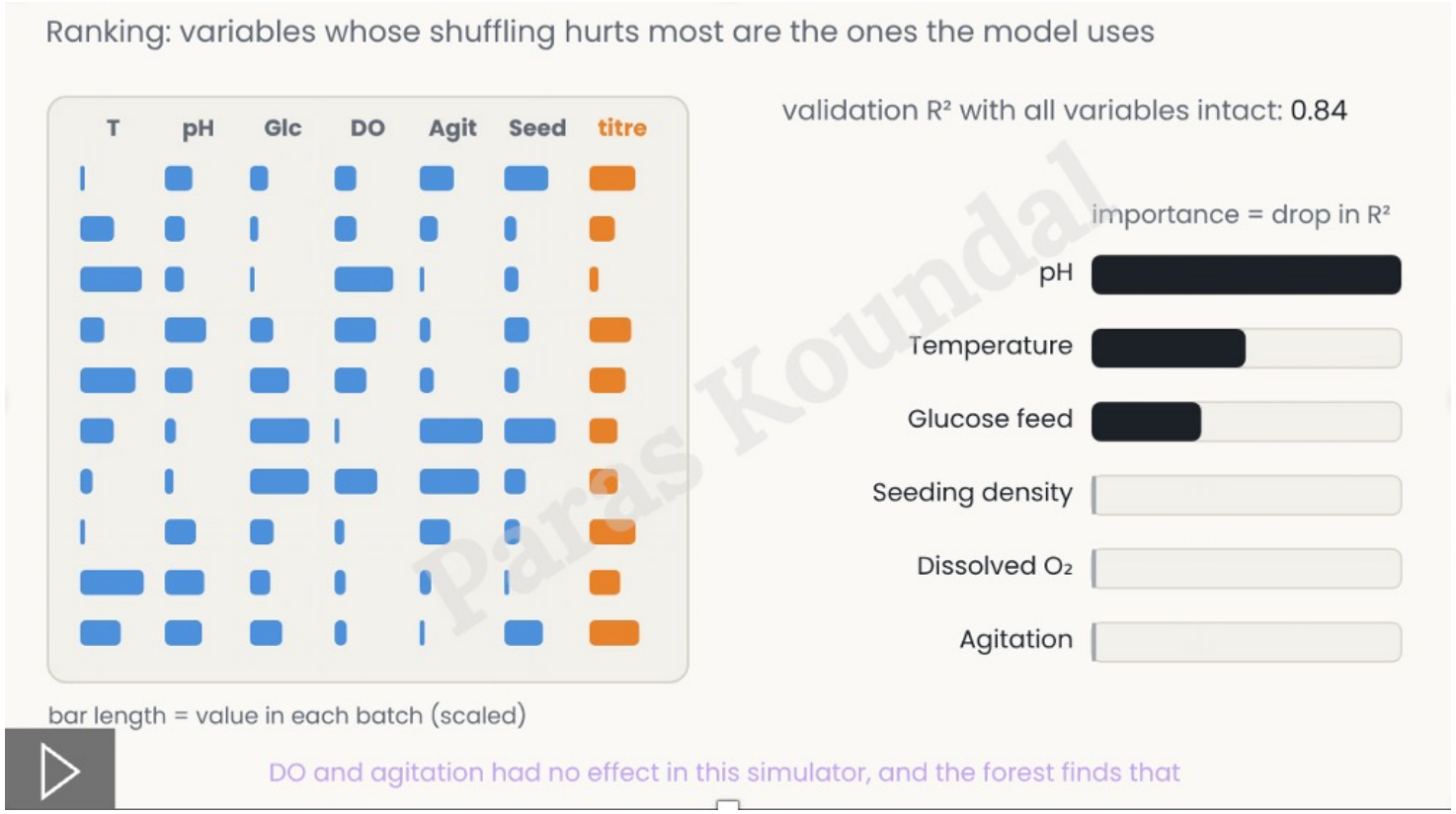
(1) A forest predicts titre from 6 simulated process variables; validation $R^2 \approx 0.84$.

(2) One column at a time is shuffled in the validation batches (watch the highlighted column scramble).

(3) Shuffling pH hurts most; temperature and glucose feed matter; DO and agitation hardly at all.

(4) At the end the bars are sorted into a ranking.

Key idea. The simulator gave DO and agitation **no effect**, and the forest finds exactly that, without being told.



Random forest: a practical recipe

setting	what it controls	sensible starting point
n_estimators	number of trees	a few hundred; more is never worse, only slower
max_features	features tried at each split (m)	$\sqrt{p}$ (classification), $p/3$ (regression); tune
max_depth / min_samples_leaf	size of each tree	usually grow deep; limit if data are noisy
oob_score	compute out-of-bag score	True: a free validation estimate

When to use. Random forests are a strong, robust **first model** for tabular data: little tuning, hard to break, honest OOB estimate.

Watch out. They cannot predict outside the range of the training data: a forest never predicts a titre higher than the highest titre it has seen.

03

Boosting

Learning from mistakes, one small tree at a time



Intuition: correcting your aim, shot by shot

Putting in golf

(1) First putt: roughly towards the hole. It stops short.

(2) Second putt: aimed at the **remaining distance**, not from the start.

(3) Third putt: corrects the small error left over.



Boosting does the same

(1) Start with a crude prediction (the average).

(2) Fit a small tree to the **errors that remain**.

(3) Add a fraction of it. Repeat many times.

Key idea. Each new tree is trained on what the current model still gets wrong. The final model is the **sum** of all the small trees.

Random forest vs boosting

Random forest

- trees built **independently**, in parallel
- **deep** trees (low bias, high variance)
- combined by **averaging**
- mainly reduces **variance**
- adding trees never overfits

Boosting

- trees built **in sequence**, each depends on the previous
- **shallow** trees, depth 2–6 (high bias, low variance)
- combined by **adding**, scaled by a learning rate η
- mainly reduces **bias**
- too many rounds **can** overfit

$$F(x) = F_0 + \eta \cdot h_1(x) + \eta \cdot h_2(x) + \ldots + \eta \cdot h_m(x)$$



Gradient boosting by hand: 4 measurements

day	measured cells (y)	F_0	residual $r = y - F_0$	tree h_1	$F_1 = F_0 + 0.5 \cdot h_1$	new residual
2	1	6	−5	−3.5	4.25	−3.25
4	4	6	−2	−3.5	4.25	−0.25
6	9	6	+3	+3.5	7.75	+1.25
8	10	6	+4	+3.5	7.75	+2.25

(1) F_0 = average of y = (1 + 4 + 9 + 10)/4 = **6**.

(2) Residuals: what F_0 gets wrong.

(3) Best one-question tree on the residuals: "day ≤ 5?" → leaves −3.5 and +3.5 (means of the residuals).

(4) Add half of it (η = 0.5).

mean squared error

F_0: 13.5

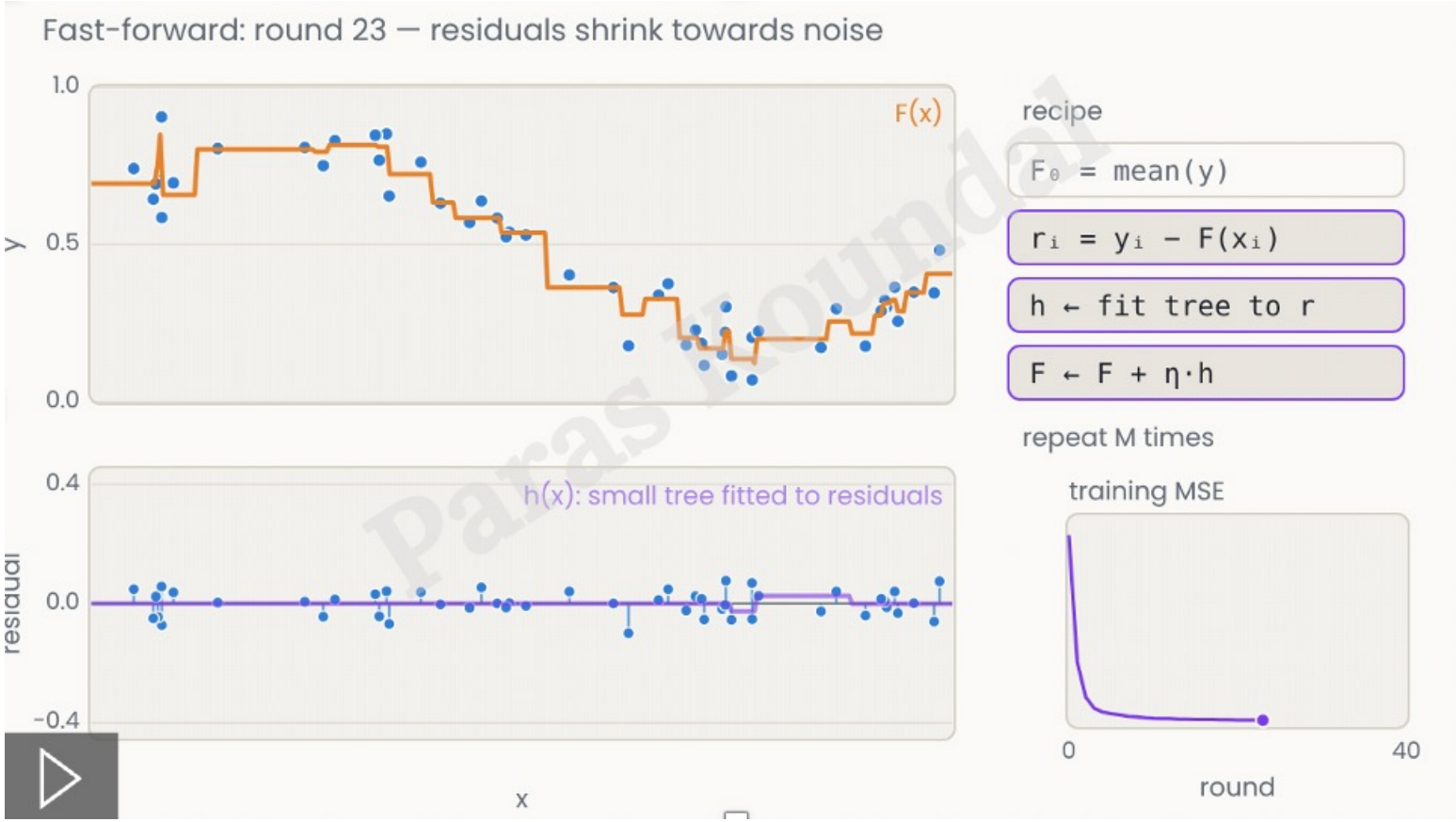
F_1: 4.31

F_2: 1.67

F_3: 0.58

Gradient boosting step by step

A small tree (red steps) is fitted to the residuals; then η·h is added to F, and F moves towards the data.



Key idea. Every round answers one question: **where is the model still too high or too low?** The new tree is the answer; adding it fixes part of the error.

Bioprocess example: boosting learns a growth curve

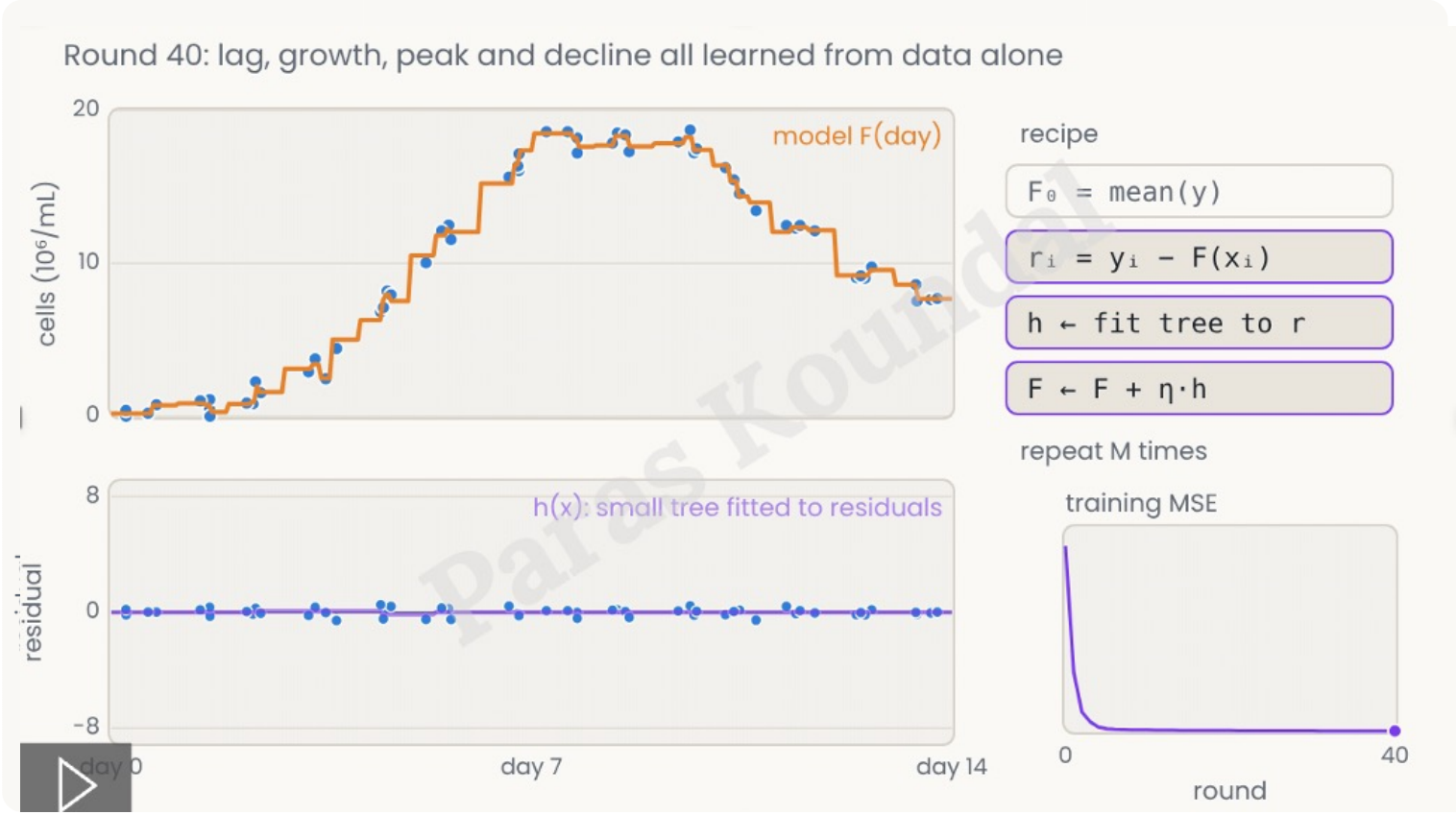
(1) Simulated viable-cell counts sampled through a 14-day culture.

(2) Round 0 predicts the same average every day: too high early, too low at the peak.

(3) Each round, a small tree learns “where is the model too high, where too low?” and a fraction is added.

(4) After 40 rounds the lag, growth, peak and decline are all captured.

Key idea. Boosting knows no biology and no growth equation. It learns the shape only by repeatedly fitting what is still wrong. The result is a step function because it is a sum of trees.



Why is it called “gradient” boosting?

Gradient = slope

The gradient of the loss with respect to a prediction says: **if I nudge this prediction up, does the loss go up or down, and how fast?**

Stepping against the gradient (downhill) reduces the loss.

For squared error

loss = ½ $(y - F)^2$

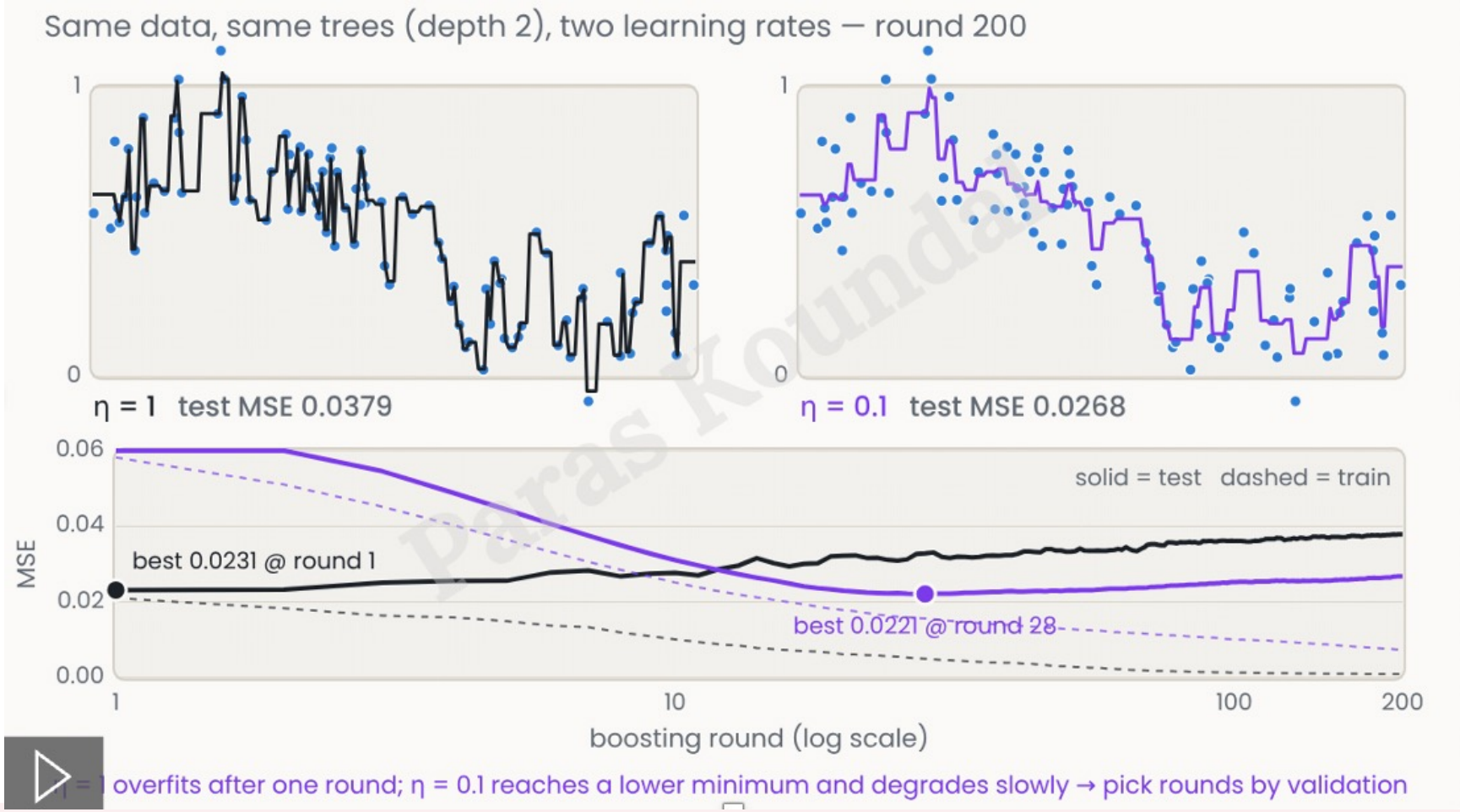
$-\,d\,\text{loss} / dF = y - F$

= **exactly the residual!**

loss	what each tree is fitted to (negative gradient)
squared error	$y - F$ (the residual)
absolute error	$\text{sign}(y - F)$: just +1 or −1
log-loss (classification)	$y - p$ (label minus predicted probability)

Key idea. Fitting trees to residuals = taking steps downhill on the loss, but the thing being adjusted is the whole **prediction function**. Lecture 2 does the same with the numbers inside a neural network.

The learning rate η: small steps generalise better



Key idea. Small η + many rounds + **early stopping** (stop when validation error stops improving) is the standard recipe.

04

XGBoost

Gradient boosting, regularised and engineered



XGBoost in plain words: three upgrades

1

Penalise complexity

The objective it minimises includes a cost for every extra leaf (γ) and for large leaf values (λ). Simpler trees are preferred unless complexity really pays off.

2

Use curvature too

Ordinary boosting uses only the slope of the loss. XGBoost also uses its curvature, which tells it **how far** to step, not only which way.

3

Engineering

Fast approximate split search, built-in handling of missing values, parallel computation. Large data sets train quickly.

Key idea. Same model as before: a sum of small trees. What changes is the **loss being minimised** and **how each tree is chosen**.

Four tree methods side by side

method	trees	combined by	mainly reduces	key settings
Decision tree	one	—	trade-off via depth	max_depth, min_samples_leaf
Random forest	many deep, independent	average / vote	variance	n_estimators, max_features
Gradient boosting	many shallow, in sequence	sum, scaled by η	bias	learning_rate, rounds, depth
XGBoost	many shallow, in sequence	sum, Newton steps	bias, regularised	+ λ, γ, subsampling

First try. Start with a **random forest**: robust, little tuning, OOB estimate for free.

Then. Then try **gradient boosting / XGBoost** with early stopping: usually the most accurate.

For explanation. Use a single **shallow tree** when you need rules people can read and check.

Boosting already did gradient descent, on predictions.

Next lecture: gradient descent on the **numbers inside a model** (its weights), and how backpropagation computes those gradients for a neural network.

Hands-on: everything today in a few lines of Python

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier

X_tr, X_va, y_tr, y_va = train_test_split(X, y, test_size=0.25)
tree = DecisionTreeClassifier(max_depth=3).fit(X_tr, y_tr)
rf   = RandomForestClassifier(n_estimators=500, oob_score=True)
rf.fit(X_tr, y_tr)
xgb  = XGBClassifier(n_estimators=2000, learning_rate=0.05, max_depth=4,
                     early_stopping_rounds=50)
xgb.fit(X_tr, y_tr, eval_set=[(X_va, y_va)])
print(rf.oob_score_, xgb.score(X_va, y_va))
```

Try it

(1) Plot validation accuracy against max_depth: find the sweet spot.

(2) Compare rf.oob_score_ with the validation score.

(3) Change learning_rate; note xgb.best_iteration.

(4) Print sklearn.tree.export_text(tree) and read the rules.

Further reading

source	what it is
Breiman, Friedman, Olshen & Stone (1984)	Classification and Regression Trees: the CART book.
Breiman (2001)	Random Forests. Machine Learning 45: 5–32.
Friedman (2001)	Greedy function approximation: a gradient boosting machine. Annals of Statistics 29: 1189–1232.
Chen & Guestrin (2016)	XGBoost: a scalable tree boosting system. KDD 2016.
Hastie, Tibshirani & Friedman	The Elements of Statistical Learning, ch. 9, 10, 15 (free PDF online).
scikit-learn user guide	sections on decision trees and ensembles: clear, practical, with examples.